2bReady

COMPLY. SCALE. LEAD.

MVP Development Proposal

Technology Proposal • Contract & Payment Terms • Development Plan • Performance, Security & Scalability • ERD • Landing Page

Engineered for High Performance • Strong Security • Enterprise Scalability

Proposed Stack: Laravel API • Next.js + shadcn/ui + Tailwind CSS • PostgreSQL (ULID)

Prepared for: 2bReady
Prepared by: Development Partner
Date: July 2026 • Version 2.1

Table of Contents

1. Technology Proposal

1.1 Overview & Objectives

This proposal defines the technical foundation, engineering standards, delivery plan, commercial terms, and data model for building the 2bReady MVP — a compliance-readiness platform connecting companies, third-party auditors, and administrators through a guided compliance journey, secure document vault, audit workflow, and trust-scoring engine.

The stack below is selected for fast MVP delivery without sacrificing the code quality, security, and scalability an enterprise SaaS product requires as it grows past MVP. Three engineering priorities run through every layer of this proposal: high performance under real-world load, strong security for the compliance data 2bReady is entrusted with, and a scale-out architecture that grows with tenant count without a rewrite — detailed in full in Section 4.

1.2 Proposed Technology Stack

Layer	Technology	Purpose
Backend API	Laravel 11 (PHP 8.3)	REST/JSON API, business logic, RBAC, queue orchestration, audit scoring engine
Frontend	Next.js 14 (App Router, TypeScript)	Admin back office + company/auditor portals, SSR for fast first paint & SEO-safe public pages
UI Library	shadcn/ui + Tailwind CSS	Accessible, themeable component primitives (Radix-based); consistent design system, no vendor lock-in
Database	PostgreSQL 16	Primary relational store; JSONB for flexible feature/config fields; row-level constraints for multi-tenancy
Primary Keys	ULID (26-char, Crockford Base32)	Sortable-by-creation, URL-safe, collision-resistant, avoids sequential-ID enumeration attacks
Auth	Laravel Sanctum + spatie/laravel-permission	SPA token auth for Next.js, cookie/CSRF safe; role & permission-based access control
Cache / Queue	Redis + Laravel Horizon	Session/cache store, job queue for score calculation, notifications, document processing
File Storage	S3-compatible object storage	Compliance documents, certificates, trust-badge assets — never stored on app servers
Realtime	Laravel Reverb (WebSockets)	Live in-app notifications, audit status updates
Search (phase 2)	Laravel Scout + Meilisearch	Fast company/document/audit search once catalog scale requires it
API Docs	Scramble (OpenAPI 3)	Auto-generated, always-current API reference for frontend & integrators
Testing	Pest (PHP) / Jest + RTL / Playwright	Unit, feature, component, and end-to-end test coverage
CI/CD	GitHub Actions	Lint → static analysis → test → build → deploy pipeline on every PR and merge
Containerization	Docker + docker-compose	Environment parity across local, staging, and production
Observability	Laravel Telescope (dev), Sentry, Laravel Pulse	Error tracking, performance monitoring, queue/health dashboards

1.3 High-Level Architecture

The system follows a decoupled architecture: Next.js is the presentation layer only and communicates with Laravel exclusively through a versioned, token-authenticated REST API. This keeps the API reusable for future mobile apps or third-party integrations (e.g. auditor marketplace partners).

- Next.js (App Router) renders the Back Office, Company Portal, and Auditor Portal as a single codebase with route groups per role.
- All data reads/writes go through /api/v1/* — no direct DB access from the frontend, ever.
- Laravel handles authentication, authorization (RBAC), validation, domain logic, compliance-score calculation, and dispatches background jobs (email/SMS notifications, document expiry checks, score recalculation).
- PostgreSQL is the single source of truth; Redis backs cache, sessions, and the Horizon queue; documents live in object storage referenced by path/URL only.
- A scheduled worker (Laravel Scheduler + Queue) runs nightly jobs: document expiry sweeps, compliance-score recompute, audit SLA reminders.

1.4 Why This Stack

- Laravel gives enterprise-grade building blocks out of the box — queues, policies, form-request validation, job batching, rate limiting — reducing time spent on infrastructure plumbing.
- Next.js + shadcn/ui gives a modern, accessible, highly customizable admin UI without being locked into a component vendor; shadcn ships source into the repo, so the team owns and can modify every component.
- PostgreSQL + ULID is the right fit for a multi-tenant B2B platform: strong relational integrity for audit/compliance data, JSONB for flexible package/feature configs, and ULIDs that are sortable (useful for activity feeds/audit logs) yet non-guessable (unlike auto-increment IDs) and safe to expose in URLs and API payloads.
- Both frameworks have mature ecosystems, large hiring pools, and long-term support — reducing key-person and maintenance risk post-handover.

1.5 Enterprise-Grade, Reusable Project Structure & Design Patterns

Backend and frontend ship as two independently deployable repositories, kept in lockstep by a single generated contract: Laravel's OpenAPI spec (Scramble) is the source of truth, and types/api.generated.ts on the frontend is regenerated from it — never hand-edited. This keeps the two repos loosely coupled operationally (separate deploys, separate CI) while remaining tightly coupled on the one thing that must never drift: the API contract.

Backend — 2bready-api (Laravel, Domain-Oriented, Interface-Driven)

Every domain folder is self-contained: its own Actions, Models, DTOs, Enums, and — only where genuinely needed — its own Contracts, Repositories, Services, and QueryFilters. Not every domain gets every subfolder; a subfolder is added only when a real requirement earns it (see the rule below the tree), so the structure stays honest rather than cargo-culted.

```
2bready-api/
├─ app/
│  ├─ Domain/
│  │  ├─ Company/
│  │  │  ├─ Actions/
│  │  │  │  ├─ CreateCompanyAction.php
│  │  │  │  ├─ SuspendCompanyAction.php
│  │  │  │  └─ UpdateCompanyProfileAction.php
│  │  │  ├─ Models/
│  │  │  │  └─ Company.php
│  │  │  ├─ DTOs/
```

```
│   │   │   │   └── CompanyData.php
│   │   │   ├── Enums/
│   │   │   │   └── CompanyStatus.php
│   │   │   ├── Events/
│   │   │   │   ├── CompanyCreated.php
│   │   │   │   └── CompanyActivated.php
│   │   │   ├── Listeners/
│   │   │   │   └── SendCompanyWelcomeNotification.php
│   │   │   ├── Policies/
│   │   │   │   └── CompanyPolicy.php
│   │   │   ├── QueryFilters/
│   │   │   │   └── CompanyListFilter.php        (status, package, search — composable, keeps the index
Action thin)
│   │   │   ├── Repositories/
│   │   │   │   ├── CompanyRepositoryInterface.php
│   │   │   │   └── EloquentCompanyRepository.php
│   │   │   └── Services/
│   │   │       └── ComplianceScoreService.php
│   │   │
│   │   ├── User/
│   │   │   ├── Actions/          (CreateUserAction, AssignRoleAction, ResetPasswordAction)
│   │   │   ├── Models/           (User.php)
│   │   │   ├── DTOs/             (UserData.php)
│   │   │   ├── Enums/            (UserStatus.php)
│   │   │   └── Policies/         (UserPolicy.php)
│   │   │
│   │   ├── Package/
│   │   │   ├── Actions/          (CreatePackageAction, UpdatePackageFeaturesAction)
│   │   │   ├── Models/           (Package.php, Subscription.php)
│   │   │   └── DTOs/             (PackageData.php)
│   │   │
│   │   ├── Payment/
│   │   │   ├── Actions/          (RecordPaymentAction, ApprovePaymentAction)
│   │   │   ├── Models/           (Payment.php)
│   │   │   ├── Services/
│   │   │   │   ├── PaymentGatewayService.php      (adapter around the gateway SDK)
│   │   │   │   └── InvoiceGeneratorService.php
│   │   │   ├── Contracts/
│   │   │   │   └── PaymentGatewayInterface.php    (so gateways are swappable)
│   │   │   ├── Gateways/
│   │   │   │   ├── StripeGateway.php
│   │   │   │   └── ManualBankTransferGateway.php
│   │   │   └── Enums/            (PaymentStatus.php, PaymentMethod.php)
│   │   │
│   │   ├── Journey/
│   │   │   ├── Actions/          (ActivateJourneyAction, UnlockNextLevelAction)
│   │   │   ├── Models/           (Journey.php, JourneyTemplate.php, JourneyLevel.php, Milestone.php)
│   │   │   ├── Services/
│   │   │   │   └── MilestoneUnlockRuleEngine.php  (strategy pattern)
│   │   │   └── Enums/            (JourneyStatus.php, JourneyLevelCode.php)
│   │   │
│   │   ├── Document/
│   │   │   ├── Actions/          (UploadDocumentAction, VerifyDocumentAction)
│   │   │   ├── Models/           (Document.php, DocumentTemplate.php)
│   │   │   ├── Jobs/             (ScanDocumentForMalwareJob.php, CheckDocumentExpiryJob.php)
│   │   │   └── Enums/            (DocumentStatus.php)
│   │   │
│   │   ├── Audit/
│   │   │   ├── Actions/          (AssignAuditorAction, ApproveAuditAction, RejectAuditAction)
│   │   │   ├── Models/           (Audit.php, Auditor.php)
│   │   │   ├── QueryFilters/
│   │   │   │   └── PendingAuditQueueFilter.php    (auditor workload, priority, SLA — same pattern as
Company)
│   │   │   ├── Services/
│   │   │   │   └── ComplianceScoreCalculator.php
│   │   │   └── Enums/            (AuditDecision.php)
│   │   │
│   │   ├── TrustBadge/
│   │   │   ├── Actions/          (IssueTrustBadgeAction.php)
│   │   │   └── Models/           (TrustBadge.php)
│   │   │
│   │   ├── Notification/
│   │   │   ├── Actions/          (SendNotificationAction.php)
```

```
│   │   ├── Models/           (Notification.php)
│   │   └── Channels/         (InAppChannel.php, EmailChannel.php)
│   │
│   ├── Support/
│   │   ├── Actions/          (CreateTicketAction, ReplyToTicketAction)
│   │   └── Models/           (SupportTicket.php, TicketMessage.php)
│   │
│   ├── Sop/
│   │   ├── Actions/          (PublishSopAction, AcknowledgeSopAction)
│   │   └── Models/           (Sop.php, SopAcknowledgement.php)
│   │
│   └── Shared/
│       └── ValueObjects/
│           └── Money.php                    (integer-cents; used by Payment + Package pricing)
│
├── Http/
│   ├── Controllers/Api/V1/
│   │   ├── CompanyController.php
│   │   ├── UserController.php
│   │   ├── PackageController.php
│   │   ├── PaymentController.php
│   │   ├── JourneyController.php
│   │   ├── DocumentController.php
│   │   ├── AuditController.php
│   │   ├── AuditorController.php
│   │   ├── NotificationController.php
│   │   ├── SupportTicketController.php
│   │   ├── SopController.php
│   │   ├── ReportController.php
│   │   └── AuthController.php
│   ├── Requests/Api/V1/
│   │   ├── Company/StoreCompanyRequest.php, UpdateCompanyRequest.php
│   │   ├── Payment/StorePaymentRequest.php
│   │   └── ...                              (one per module, mirrors the domain list above)
│   ├── Resources/Api/V1/
│   │   ├── CompanyResource.php, CompanyCollection.php
│   │   ├── UserResource.php, PaymentResource.php, JourneyResource.php, AuditResource.php, ...
│   ├── Middleware/
│   │   ├── EnsureCompanyIsActive.php
│   │   ├── ScopeToCompany.php               (request-level tenant scoping)
│   │   └── ForceJsonResponse.php
│   └── Kernel.php
│
├── Models/                                  (kept empty — every real model lives under Domain/*/Models)
├── Policies/                                (empty — actual policy logic lives in Domain/*/Policies)
│
├── Providers/
│   ├── AppServiceProvider.php
│   ├── AuthServiceProvider.php              (registers all Policies)
│   ├── EventServiceProvider.php             (registers all Event → Listener bindings)
│   ├── RepositoryServiceProvider.php        (binds Interfaces → Eloquent implementations)
│   └── HorizonServiceProvider.php
│
├── Console/Commands/
│   ├── RecalculateComplianceScoresCommand.php
│   └── SweepExpiredDocumentsCommand.php
│
├── Exceptions/
│   ├── Handler.php
│   ├── DomainException.php                  (base class for all business-rule exceptions)
│   ├── InsufficientComplianceScoreException.php
│   └── DocumentExpiredException.php
│
└── Support/                                 (framework-agnostic helpers, no business logic)
    ├── Concerns/
    │   ├── HasUlid.php                      (trait: auto-generates ULID primary key)
    │   └── BelongsToCompany.php             (trait + global scope — THE tenant-isolation boundary;
    │                                         applied to every tenant-scoped model, enforced at query
time)
    └── ApiResponse.php                      (standard {data, meta} / {message, errors} envelope)

├── database/
│   ├── migrations/                          (one file per table; company_id + ulid on every tenant
```

```
table)
│   ├─ seeders/
│   │   ├─ DatabaseSeeder.php
│   │   ├─ RolePermissionSeeder.php
│   │   └─ DemoDataSeeder.php
│   └─ factories/
│       ├─ CompanyFactory.php, UserFactory.php, ...   (one per domain model, used by seeders and tests)
│
├─ routes/
│   ├─ api.php                              (thin — groups by prefix('v1'), delegates to domain route
files)
│   └─ api/
│       ├─ company.php, payment.php, journey.php, document.php, audit.php
│
├─ config/
│   └─ compliance.php                       (journey levels, score thresholds — business config, not
code)
│
├─ tests/
│   ├─ Feature/Api/V1/
│   │   ├─ CompanyTest.php                  (happy path + validation + authorization per endpoint)
│   │   └─ PaymentTest.php, ...
│   ├─ Unit/Domain/
│   │   ├─ Audit/ComplianceScoreCalculatorTest.php
│   │   └─ Journey/MilestoneUnlockRuleEngineTest.php
│   └─ Pest.php
│
├─ .env.example
├─ phpstan.neon                             (Larastan config, CI-gated)
├─ pint.json                                (PSR-12 formatting rules)
└─ composer.json
```

Backend Structure Rules

- Repository + Contracts is added to a domain only when there's a real reason to abstract persistence or swap an implementation — Company (mockable in tests, future caching layer) and Payment (multiple gateways behind one interface) qualify; a straightforward CRUD domain like Support does not, and skips both folders rather than carrying dead abstraction.
- Services/ holds orchestration logic reused by more than one Action (ComplianceScoreService, MilestoneUnlockRuleEngine, PaymentGatewayService); a domain with only single-step Actions has no Services/ folder at all.
- QueryFilters/ is added to any domain with a non-trivial list/search endpoint (Company directory, Audit review queue) — keeps filtering composable and out of both the Controller and the Action.
- Multi-tenancy is enforced in exactly one place: the BelongsToCompany trait/global scope in app/Support/Concerns/ — applied to every tenant-scoped model. This is the single most safety-critical file in the codebase, deliberately kept framework-level and not duplicated per-domain, so there is one boundary to audit, not fifteen.
- Shared cross-domain concepts (Money) live in Domain/Shared/, signalling they're a domain concept multiple modules depend on — not a generic framework helper.

Backend Design Patterns Applied

- Repository Pattern — Company and Payment define a *RepositoryInterface/*GatewayInterface (Contracts/) with a concrete implementation, bound once in RepositoryServiceProvider. Business code depends only on the interface.
- Action / Service Pattern — Actions are single-responsibility, invokable, and called directly from Controllers; Services hold logic more than one Action needs (score calculation, unlock evaluation, gateway orchestration), so it's never duplicated or trapped inside one Action.
- Strategy Pattern — PaymentGatewayInterface with StripeGateway / ManualBankTransferGateway implementations, and MilestoneUnlockRuleEngine for pluggable unlock rules per journey level.

- DTO Pattern (spatie/laravel-data) — typed objects cross layer boundaries instead of raw arrays, catching shape errors at development time.
- Observer / Event-Listener Pattern — CompanyCreated, CompanyActivated and similar domain events decouple side effects (welcome notifications, score recalculation) from the Action that triggered them.
- Specification / Query Filter Pattern — CompanyListFilter and PendingAuditQueueFilter encapsulate composable list filtering, reusable and independently testable.
- Global Scope Pattern for multi-tenancy — BelongsToCompany enforces company_id isolation at the query layer for every tenant model, so a coding mistake in one Action can't leak another company's rows.
- Dependency Injection throughout — everything is resolved via the container and type-hinted against an interface where one exists, never instantiated with new inside business logic.

Frontend — 2bready-web (Next.js, Feature-Sliced Architecture)

Routing stays a thin shell — layouts and page.tsx files only. All data-fetching, validation, and UI for a capability live together inside that capability's feature slice, so a domain can be understood, tested, or handed to another engineer without hunting across the tree.

```
2bready-web/
├─ src/
│  ├─ app/
│  │  ├─ (marketing)/                    (public, statically generated)
│  │  │  ├─ layout.tsx
│  │  │  ├─ page.tsx
│  │  │  └─ pricing/page.tsx
│  │  │
│  │  ├─ (auth)/
│  │  │  ├─ login/page.tsx
│  │  │  ├─ forgot-password/page.tsx
│  │  │  └─ layout.tsx
│  │  │
│  │  ├─ (backoffice)/                   (role-guarded: admin, finance, staff)
│  │  │  ├─ layout.tsx                   (sidebar shell, role-based nav)
│  │  │  ├─ dashboard/page.tsx
│  │  │  ├─ companies/page.tsx, [companyId]/page.tsx
│  │  │  ├─ users/page.tsx, packages/page.tsx, payments/page.tsx
│  │  │  ├─ journey-builder/page.tsx, document-templates/page.tsx, documents/page.tsx
│  │  │  ├─ audits/page.tsx, auditors/page.tsx
│  │  │  ├─ notifications/page.tsx, support/page.tsx, reports/page.tsx, settings/page.tsx
│  │  │
│  │  ├─ (company-portal)/               (role-guarded: company users)
│  │  │  ├─ layout.tsx
│  │  │  ├─ dashboard/page.tsx, journey/page.tsx, documents/page.tsx
│  │  │  ├─ data-room/page.tsx, sops/page.tsx
│  │  │
│  │  ├─ (auditor-portal)/
│  │  │  ├─ layout.tsx
│  │  │  └─ assignments/page.tsx
│  │  │
│  │  ├─ api/                            (server-only routes — auth callback / webhook relays;
│  │  │                                   NOT business logic, that stays in Laravel)
│  │  ├─ layout.tsx                      (root layout: fonts, providers)
│  │  └─ global-error.tsx
│  │
│  ├─ features/
│  │  ├─ companies/
│  │  │  ├─ api/
│  │  │  │  ├─ useCompanies.ts           (TanStack Query hook: list)
│  │  │  │  ├─ useCompany.ts             (TanStack Query hook: detail)
│  │  │  │  ├─ useCreateCompany.ts       (mutation hook)
│  │  │  │  └─ companies.api.ts          (typed fetch functions, thin wrapper over lib/api-client)
│  │  │  ├─ components/
│  │  │  │  ├─ CompanyTable.tsx, CompanyForm.tsx, CompanyStatusBadge.tsx
│  │  │  ├─ schemas/
│  │  │  │  └─ company.schema.ts         (Zod — mirrors backend Form Request rules)
│  │  │  └─ types.ts                     (generated/augmented from the OpenAPI spec)
│  │  │
```

```
│   │   ├── users/, packages/, payments/        (same api/ + components/ + schemas/ + types.ts shape)
│   │   ├── journey/
│   │   │   └── components/
│   │   │       ├── JourneyBuilderCanvas.tsx
│   │   │       └── MilestoneUnlockRuleEditor.tsx
│   │   ├── documents/, audits/, auditors/, notifications/, support/, sops/, reports/
│   │   ├── auth/
│   │   │   ├── api/useLogin.ts, useLogout.ts, useCurrentUser.ts
│   │   │   └── components/LoginForm.tsx
│   │   └── marketing/
│   │       ├── components/    (Hero, HowItWorks, FeatureGrid, PricingTable, Faq, Testimonials)
│   │       ├── animations/    (shared Framer Motion variants)
│   │       └── content.ts
│   │
│   ├── components/
│   │   ├── ui/                                (shadcn/ui primitives — untouched from the CLI generator)
│   │   └── shared/                            (composed-but-generic: DataTable, PageHeader, EmptyState,
│   │                                           ConfirmDialog, StatusBadge — used across multiple
features)
│   │
│   ├── hooks/
│   │   ├── use-debounce.ts
│   │   ├── use-permission.ts                  (client-side check, mirrors backend Policy names)
│   │   └── use-pagination.ts
│   │
│   ├── lib/
│   │   ├── api-client.ts                      (single fetch wrapper: base URL, auth header, error
envelope
│   │   │                                       parsing — every feature's api/ file goes through this)
│   │   ├── auth.ts                            (session/token handling)
│   │   ├── query-client.ts                    (TanStack Query client + default options)
│   │   ├── utils.ts                           (cn(), formatMoney(), formatDate() ...)
│   │   └── constants.ts
│   │
│   ├── store/
│   │   └── ui-store.ts                        (Zustand — sidebar collapsed, active theme; NOT server data
—
│   │                                           server data always lives in TanStack Query, never
duplicated here)
│   │
│   ├── types/
│   │   ├── api.generated.ts                   (openapi-typescript output — regenerated from Laravel's
│   │   │                                       Scramble OpenAPI spec; never hand-edited)
│   │   └── index.ts
│   │
│   └── middleware.ts                          (route guards: redirect unauthenticated users,
│                                                 role-based route protection before page render)
│
├── tests/
│   ├── unit/                                  (Vitest + React Testing Library — components, hooks)
│   └── e2e/                                   (Playwright — login, create-company, payment, audit-
approval flows)
│
├── .env.example
├── tailwind.config.ts
├── components.json                            (shadcn/ui config)
├── eslint.config.js
└── package.json
```

Frontend Design Patterns Applied

- Repository-style API layer — every feature's api/ folder is the only place that calls the network, always through the shared lib/api-client.ts. Components consume typed hooks (useCompanies, useCreateCompany), never fetch directly — swapping REST for GraphQL later touches one folder, not every component.
- Container / Presenter split — feature api/ hooks own data-fetching; components/ (CompanyTable, CompanyForm) stay presentational and reusable independent of where the data comes from.
- Compound Components — JourneyBuilderCanvas and MilestoneUnlockRuleEditor compose several sub-components sharing implicit context for a complex, stateful editing surface.

- Shared generic components (components/shared/: DataTable, PageHeader, EmptyState, ConfirmDialog, StatusBadge) are deliberately separated from components/ui/ primitives — ui/ stays an untouched shadcn mirror; shared/ is where the team's own reusable composition lives.
- Adapter Pattern — types/api.generated.ts adapts the backend's OpenAPI contract into consumable TypeScript types; hand-written types/index.ts augments it without ever editing the generated file.
- Minimal, explicit global state — store/ui-store.ts (Zustand) is scoped to UI-only state (sidebar, theme) by convention and comment; every piece of server data lives in TanStack Query, so there is exactly one source of truth for anything fetched from the API.
- Mirrored authorization — use-permission.ts mirrors backend Policy names 1:1, so a frontend permission check can never silently drift from what the API actually enforces; middleware.ts applies route-level guards before a page even renders.

API Contract Conventions

- All endpoints versioned: /api/v1/..., allowing v2 to be introduced without breaking existing clients.
- Consistent success envelope: { data, meta } and error envelope: { message, errors } for all endpoints, parsed once in lib/api-client.ts on the frontend.
- Pagination via cursor or page-based Laravel pagination with meta.total, meta.per_page, links.next.
- Standard HTTP status codes throughout (422 validation, 403 authorization, 404 not found, 409 conflict, 429 rate-limited).
- The OpenAPI spec is generated once (Scramble) and consumed twice: openapi-typescript regenerates types/api.generated.ts for the frontend, and docs/api/ publishes the human-readable reference — one schema, zero manual syncing between repos.

1.6 Development Rules & Coding Standards

Language & Style

- PHP: PSR-12, enforced automatically by Laravel Pint on every commit (pre-commit hook + CI gate).
- PHP: strict_types=1 in every file; PHPStan / Larastan at level 6+ run in CI — no new code may introduce PHPStan errors.
- TypeScript: strict mode on, no any without an inline justification comment; ESLint (Next.js core-web-vitals + custom rules) + Prettier enforced via pre-commit and CI.
- No commented-out code, no console.log/dd() left in committed code — caught by lint rules.

Git & Review Workflow

- Conventional Commits (feat:, fix:, chore:, refactor:, docs:) — enables auto-generated changelogs.
- Trunk-based development: short-lived feature branches (feature/journey-unlock-logic), max ~2-3 days old before merge.
- Every PR requires: passing CI (lint + static analysis + tests), at least 1 reviewer approval, and a filled-out PR template (what/why/how tested/screenshots for UI changes).
- Squash-merge to keep main history linear and readable.

Testing Requirements

- Every API endpoint ships with a Pest feature test covering the happy path, validation failures, and authorization failures (403/401).
- Domain services and score-calculation logic require unit tests — this logic determines audit outcomes and must be provably correct.

- Minimum coverage targets: 80% on Domain/ business logic, 60% overall backend, critical user flows (auth, payment, document upload, audit approval) covered end-to-end with Playwright.
- CI blocks merge if coverage drops below the configured threshold or any test fails.

Security Rules

- All input validated server-side via Form Requests — client-side validation is UX only, never trusted.
- Authorization checked via Policies/Gates on every controller action, never inferred from the UI hiding a button.
- Eloquent/query builder only — no raw SQL string concatenation; mass-assignment protected via $fillable allow-lists.
- Rate limiting on auth, payment, and document-upload endpoints; audit log (immutable) recorded for every sensitive action (login, role change, payment approval, audit decision).
- Secrets never committed — .env only, managed via the hosting provider's secret manager in staging/production.
- Automated dependency scanning (GitHub Dependabot / composer audit / npm audit) runs weekly and on every PR.
- File uploads validated by MIME + size + antivirus scan hook before being persisted to object storage; documents served via short-lived signed URLs, never public buckets.

Documentation & Definition of Done

- Every module ships a short README (purpose, key models, how to run its tests).
- Non-trivial architectural decisions recorded as lightweight ADRs (docs/adr/0001-...) so future engineers understand why, not just what.
- **A task is "Done" only when:** code merged to main, tests passing in CI, API documented, no new lint/static-analysis warnings, reviewed by a peer, and demoed (or verifiable) in the sprint review.

2. Contract & Payment Terms

2.1 Engagement Model

Fixed-price, milestone-based engagement across the 8-sprint MVP roadmap. Scope, deliverables, and acceptance criteria for each phase are fixed at contract signing; anything outside the agreed backlog is handled through the Change Request process (§2.5), not absorbed silently into the fixed price.

2.2 Payment Schedule

Phase	Milestone	% of Contract	Trigger / Deliverable
Phase 0	Kickoff & Discovery	10%	Contract signed; technical discovery, environment setup, fina backlog
Phase 1	Sprints 1–2 — Foundation	20%	Auth, Roles/RBAC, Admin Dashboard, Company Managemen complete & demoed
Phase 2	Sprints 3–5 — Core Journey	25%	Packages/Payments, Journey Builder, Document Templates & complete & demoed
Phase 3	Sprints 6–8 — Audit & Launch-Readiness	25%	Audit/Auditor Management, Notifications/Support, Reportin complete & demoed
Phase 4	UAT & Go-Live	15%	User Acceptance Testing signed off, production deployment
Phase 5	Warranty Holdback	5%	Released after the 30-day post-launch warranty period (§2.7 unresolved critical defects

2.3 Invoicing & Payment Terms

- Invoices issued at the start of each phase (except the go-live and warranty milestones, invoiced on completion) and are due Net 7 from invoice date.
- Payments via bank transfer in the currency agreed at signing (USD by default).
- Late payments beyond 14 days accrue a 1.5% per month late fee and may pause active development until resolved.
- Third-party costs are billed separately at cost, not included in the fixed price: hosting/cloud infrastructure, payment-gateway fees, SMS/email delivery credits, SSL/domain, and any paid third-party API subscriptions.

2.4 Acceptance & UAT Process

- Each phase ends with a live demo against the agreed acceptance criteria for that phase's sprints.
- The client has 5 business days to review each phase deliverable and submit written feedback; issues are triaged as either in-scope defects (fixed at no cost) or new requests (routed to Change Request).
- If no feedback is received within the review window, the deliverable is deemed accepted and the corresponding invoice becomes payable.

2.5 Change Request Policy

- Any request outside the signed backlog (new modules, redesigns of already-accepted screens, scope changes) is logged as a Change Request with an estimate before work begins.

- Change Requests are billed at an agreed hourly/day rate, separate from the fixed-price milestones, and require written sign-off before implementation starts.
- Change Requests that affect a milestone already in progress may shift that milestone's timeline; the revised date is communicated at approval time.

2.6 Intellectual Property & Licensing

- All custom source code, designs, and documentation produced under this engagement transfer to the client upon receipt of full payment for the corresponding phase.
- Open-source packages used (Laravel, Next.js, shadcn/ui, and their dependencies) remain under their respective OSS licenses; the client receives full rights to use, modify, and redistribute the custom application code built on top of them.
- The development team retains the right to reuse generic, non-client-specific components, patterns, and internal tooling built during the engagement.

2.7 Warranty & Post-Launch Support

- A 30-day warranty period begins at go-live: defects that are reproducible deviations from agreed acceptance criteria are fixed at no additional cost.
- The warranty does not cover new feature requests, third-party service outages, or issues introduced by changes made outside the delivery team.
- An optional ongoing support/maintenance retainer (monthly hours block covering bug fixes, minor enhancements, dependency upgrades, and monitoring) can be agreed separately for after the warranty period ends.

2.8 Confidentiality & Termination

- Both parties keep confidential information (source code, business data, credentials, roadmap) private during and after the engagement.
- Either party may terminate with 14 days' written notice; the client pays for all work completed and accepted up to the termination date, and receives all source code and assets delivered to that point.

3. Development Plan

3.1 Team Composition

Role	Responsibility	Allocation
Project / Delivery Manager	Sprint planning, client demos, scope & risk management, reporting	Part-time, all sprints
Backend Engineer (Laravel) × 2	API, domain services, RBAC, integrations, background jobs	Full-time, all sprints
Frontend Engineer (Next.js) × 2	Back office, company & auditor portals, shadcn/ui component work	Full-time, all sprints
UI/UX Designer	Wireframes, design system, high-fidelity screens ahead of each sprint	Part-time, Sprints 1–6
QA Engineer	Test plans, manual + automated regression, UAT support	Part-time ramping to full-time Sprints 6–8
DevOps (shared)	CI/CD, environments, infrastructure, monitoring setup	Part-time, Sprints 1 & 7–8

3.2 Methodology

Agile Scrum with 2-week sprints, matching the 8 sprints defined in the MVP roadmap. Each sprint includes: sprint planning (backlog refinement + estimation), daily standups, a mid-sprint check-in, a sprint review/demo with the client, and a retrospective.

3.3 Scope by Module

Each sprint below maps 1:1 to the MVP roadmap. Acceptance criteria are verified in the sprint review before the corresponding contract milestone is invoiced.

Sprint	Module	Key Scope / User Stories	Priority
1	Authentication, Roles, Dashboard	Login + 2FA · RBAC (roles/permissions) · Admin dashboard shell with KPI overview	★★★★
2	Company Management	Company CRUD · company profile & business details · status (active/suspend) · progress view	★★★★
3	Packages & Payment Activation	Package management · payment integration · payment verification · subscription activation	★★★★★
4	Journey Builder & Unlock Logic	Journey/level/milestone builder · score-based unlock rules · journey activation by plan	★★★★★
5	Document Templates & Upload	Template setup (required/optional/expiry) · document upload · document management	★★★★
6	Audit & Auditor Management	Auditor management/assignment · audit review & decision workflow · audit report generation	★★★★
7	Notifications & Support	Email/in-app notifications · support ticketing · help center / FAQ	★★★
8	Reporting, QA, Deployment	Reports & analytics · system/performance/security testing · production deployment & go-live	★★★★

3.4 Task Schedule

Total MVP build: 8 sprints × 2 weeks = 16 weeks, plus a 2-week UAT/hardening & go-live buffer — approximately 4.5 months end to end.

Sprint	Duration	Weeks	Focus	Payment Phase
Kickoff	1 week	W0	Discovery, environment & repo setup, backlog sign-off	Phase 0
Sprint 1	2 weeks	W1–W2	Auth, Roles, Dashboard	Phase 1
Sprint 2	2 weeks	W3–W4	Company Management	Phase 1
Sprint 3	2 weeks	W5–W6	Packages & Payment Activation	Phase 2
Sprint 4	2 weeks	W7–W8	Journey Builder & Unlock Logic	Phase 2
Sprint 5	2 weeks	W9–W10	Document Templates & Upload	Phase 2
Sprint 6	2 weeks	W11–W12	Audit & Auditor Management	Phase 3
Sprint 7	2 weeks	W13–W14	Notifications & Support	Phase 3
Sprint 8	2 weeks	W15–W16	Reporting, QA, Deployment prep	Phase 3
UAT / Go-Live	2 weeks	W17–W18	User acceptance testing, hardening, production launch	Phase 4
Warranty	4 weeks	W19–W22	Post-launch monitoring & free defect fixes	Phase 5

3.5 QA Strategy & Definition of Done

- Every sprint's demo is preceded by a QA pass: functional test cases derived from the sprint's acceptance criteria, executed manually and, where stable, automated with Playwright.
- Sprint 8 includes a dedicated security/performance pass: auth & authorization checks, rate-limit verification, load test on the payment and document-upload endpoints, and a dependency vulnerability scan.
- Definition of Done for each backlog item: code merged and reviewed, tests passing in CI, API documented, demoed to the client, no open critical/high defects.

3.6 Risks & Assumptions

- Assumes the client provides timely feedback within the 5-business-day UAT windows (§2.4) — delays here shift the overall timeline.
- Assumes third-party integrations (payment gateway, email/SMS provider) are selected and credentials available by Sprint 3.
- Any scope added mid-sprint is deferred to the next sprint or handled via Change Request, to protect already-committed milestone dates.

4. Performance, Security & Scalability Architecture

The MVP is built to survive its own success: the same architecture that ships an 8-sprint MVP is the one that carries 2bReady to thousands of companies, auditors, and documents without a rewrite. This section defines the concrete performance targets, defense-in-depth security model, and scale-out strategy applied from Sprint 1 — not retrofitted later.

4.1 Performance Engineering

Targets (SLOs)

Metric	Target	How it's protected
API p95 response time	< 250ms (reads), < 500ms (writes)	Query indexing, eager loading, Redis cache, queued side-effects
Dashboard / page load (TTFB)	< 200ms	Next.js SSR/RSC + edge caching + CDN
Document upload (≤25MB)	< 3s to acknowledge	Direct-to-storage signed uploads, processing moved to a queued job
Uptime (production)	99.9% monthly	Multi-AZ deploy, health checks, autoscaling, zero-downtime releases
Concurrent companies supported (MVP)	1,000+ tenants	Stateless API + horizontal scaling + connection pooling

Backend Performance Practices

- N+1 queries are a CI-blocking issue: Laravel Debugbar/Telescope query counts are checked in review; eager-loading (with()) is required wherever a list endpoint returns related models.
- Every foreign key and every column used in a WHERE/ORDER BY/JOIN is indexed by migration — including composite indexes for common filters (e.g. (company_id, status), (company_id, created_at) for activity feeds).
- ULIDs are stored as PostgreSQL native uuid/char(26) with a B-tree index; being time-sortable, they avoid the random-insert index fragmentation that plain UUIDv4 causes at scale.
- Redis caches expensive, slow-changing reads (compliance-score breakdowns, package/feature configs, dashboard aggregates) with explicit, event-driven invalidation — never time-only stale caches for data that drives audit decisions.
- Heavy or slow work (compliance-score recalculation, PDF/report generation, email/SMS dispatch, document virus-scan) is always dispatched to Laravel Horizon queues, never run inline on the request thread.
- List endpoints are paginated by default (max page size enforced server-side) — no unbounded 'return everything' endpoints.
- Database read/write split is architecture-ready from day one: Eloquent read/write connections are configured separately so a read replica can be added in production without an application code change.

Frontend Performance Practices

- Next.js App Router with React Server Components for data-heavy screens (dashboards, reports) — HTML is streamed, not waiting on a client-side fetch waterfall.
- Route-level code splitting and dynamic imports for heavy components (charts, the document viewer, the journey builder canvas) keep the initial JS bundle lean.

- Tailwind CSS is compiled/purged at build time — only the utility classes actually used ship to the browser, keeping CSS payload minimal regardless of shadcn/ui component count.
- TanStack Query caches and de-duplicates requests client-side, with stale-while-revalidate for non-critical data (notification counts, dashboard tiles).
- Images/certificates/trust-badge assets served through a CDN with automatic resizing/format negotiation (WebP/AVIF) rather than raw uploads.
- Core Web Vitals (LCP, CLS, INP) tracked in CI via Lighthouse budgets — a PR that regresses the budget fails the build.

4.2 Security Architecture (Defense in Depth)

As a platform that stores companies' compliance documents, audit outcomes, and trust scores, 2bReady's own security posture is itself part of the product's credibility. Security is layered so no single control failure exposes data.

Application-Layer Security

- Authentication via Laravel Sanctum (SPA token, HttpOnly + SameSite cookies) with mandatory 2FA for Admin, Finance, and Auditor roles; brute-force login protection via rate limiting + exponential lockout.
- Authorization is enforced twice: RBAC via spatie/laravel-permission at the route/policy layer, and row-level tenant scoping via a global Eloquent scope so one company's query can never touch another's rows, even on a coding mistake.
- All input validated server-side through Form Requests with strict typing; client-side validation is UX convenience only and is never trusted as a security boundary.
- Protection mapped explicitly to the OWASP Top 10: parameterized queries only (no raw SQL) for injection; output-escaped Blade/React rendering + a strict Content-Security-Policy for XSS; Sanctum CSRF tokens on all state-changing requests; mass-assignment locked down via explicit $fillable allow-lists; dependency vulnerability scanning (composer audit / npm audit / Dependabot) for known-CVE components.
- Sensitive actions (login, role change, payment approval, audit decision, document access, data-room link creation) write to an immutable audit_logs table — who, what, when, from where — itself protected from update/delete at the database role level.

Data Protection

- Encryption in transit: TLS 1.2+ enforced everywhere (HSTS enabled); encryption at rest: database-level encryption plus field-level encryption (Laravel's encrypted cast) for highly sensitive fields (e.g. banking/payment references).
- Compliance documents and trust-badge assets live in private object storage, never a public bucket; access is only ever through short-lived, signed URLs issued per-request and scoped to the requesting user's permissions.
- The Smart Data Room's external sharing links are time-limited, revocable, and optionally PIN-protected, with every access event logged to the audit trail.
- Secrets (DB credentials, API keys, signing keys) are never committed to the repository; managed via the hosting provider's secret manager, injected as environment variables at deploy time, and rotated on a defined schedule.
- Backups: automated daily encrypted PostgreSQL backups with point-in-time recovery, tested via a quarterly restore drill.

Infrastructure & Perimeter Security

- Rate limiting at both the edge (WAF/CDN) and application layer, with tighter limits on auth, payment, and upload endpoints specifically.
- A Web Application Firewall in front of the API blocks common attack signatures before they reach Laravel.

- Staging and production run on isolated networks/VPCs with least-privilege IAM — engineers do not get standing production database access; break-glass access is logged.
- File uploads are validated by MIME type and size before acceptance, then run through an antivirus/malware scan hook prior to being persisted to storage.
- Dependency and container images are scanned on every build; the CI pipeline blocks merges/deploys on critical/high vulnerabilities.
- A security review (auth flows, authorization boundaries, rate limits, upload handling) is scheduled at the end of Sprint 8 (§3.5), ahead of go-live; an external penetration test is recommended before onboarding real customer compliance data at scale.

4.3 Scalability & High Availability

Horizontal Scale-Out by Design

- The Laravel API is fully stateless — sessions/auth tokens live in Redis, not local memory — so any number of API instances can sit behind a load balancer with no sticky-session requirement.
- Next.js is deployed as a stateless edge/server runtime, independently scalable from the API tier, with static and cacheable routes served from the CDN edge rather than hitting origin at all.
- Queue workers (Laravel Horizon) scale independently from web traffic — a spike in document uploads or score recalculation queues more workers without touching the request-serving fleet.
- PostgreSQL scales vertically first (right-sized instance), then horizontally via read replicas for reporting/analytics queries, keeping the primary free for write throughput as tenant count grows.

Multi-Tenancy Growth Path

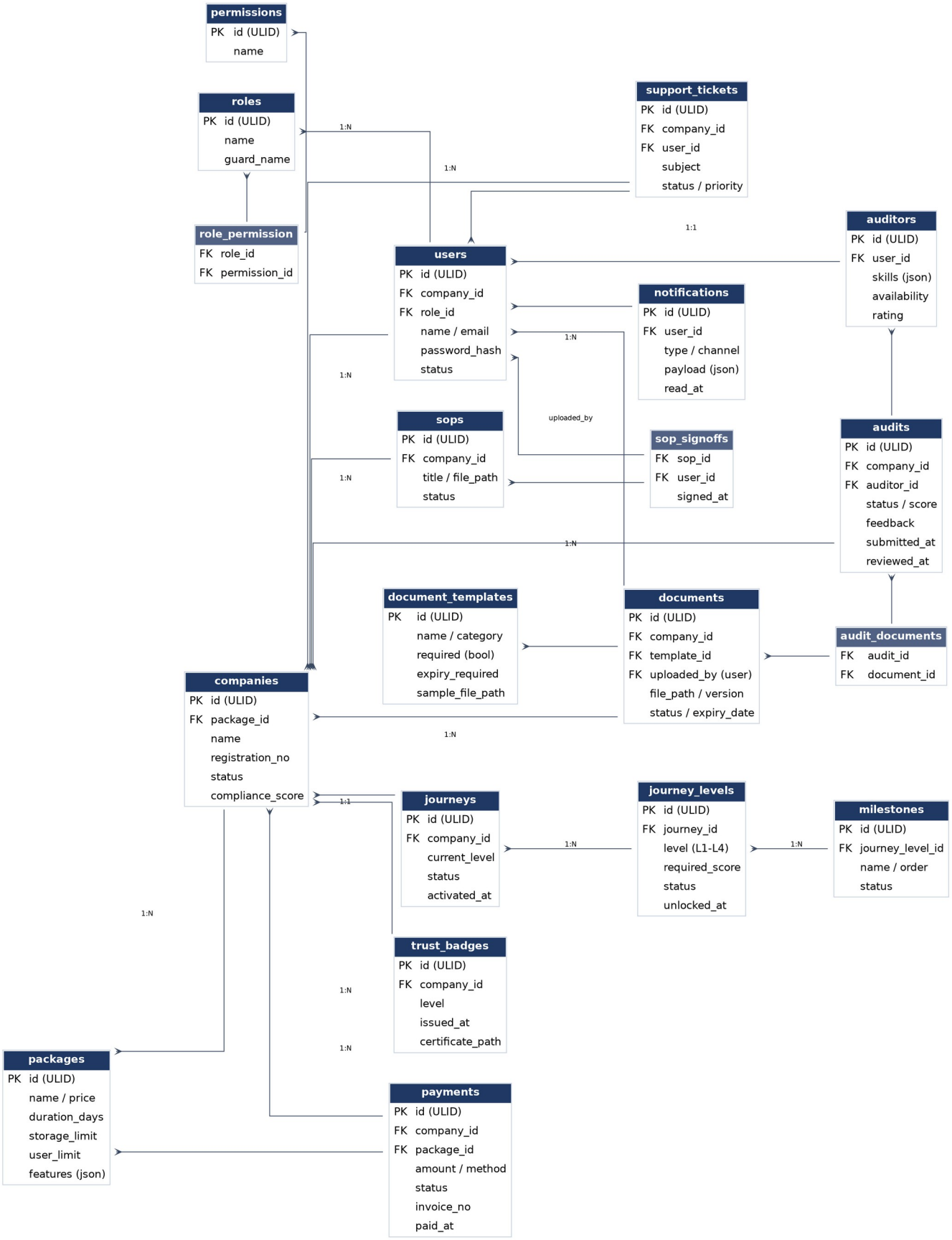
- MVP uses a shared-database, row-level tenant isolation model (company_id scoping) — the simplest, fastest-to-ship approach that still keeps a clean upgrade path.
- If a small number of large/regulated clients later require stronger isolation, PostgreSQL native table partitioning by company_id (or a dedicated schema-per-tenant for enterprise customers) can be introduced without changing the application's data-access layer, since all access already goes through the tenant-scoped Eloquent layer.
- JSONB feature/config columns on packages and companies allow new plan tiers and feature flags to ship without schema migrations, supporting fast commercial iteration post-MVP.

Reliability & Operations

- Zero-downtime deploys via a blue-green / rolling release strategy, with automated migration checks and an automated rollback path if health checks fail post-deploy.
- Health-check endpoints (/health, /health/queue, /health/db) feed the load balancer and uptime monitoring so a failing instance is pulled from rotation automatically.
- Observability stack (Sentry for errors, Laravel Pulse for app health, infrastructure metrics/log aggregation) with alerting thresholds on error rate, queue backlog depth, and p95 latency — not just server-up/down.
- Load testing (k6 or Artillery) against auth, payment, document-upload, and dashboard endpoints is run before go-live and after any major architectural change, against the SLO targets in §4.1.
- Infrastructure-as-code (Docker + a documented provisioning script/Terraform) so staging and production environments are reproducible, not hand-configured.

5. Entity Relationship Diagram (ERD)

Core MVP data model. All primary keys are ULIDs (26-character, sortable, non-enumerable identifiers) stored as PostgreSQL char(26) with a unique index; every table also carries created_at / updated_at, and soft-deletes (deleted_at) where records must be recoverable (companies, documents, audits).


permissions
PK id (ULID)
name
roles
PK id (ULID)
name
guard_name
role_permission
FK role_id
FK permission_id
support_tickets
PK id (ULID)
FK company_id
FK user_id
subject
status / priority
auditors
PK id (ULID)
FK user_id
skills (json)
availability
rating
users
PK id (ULID)
FK company_id
FK role_id
name / email
password_hash
status
notifications
PK id (ULID)
FK user_id
type / channel
payload (json)
read_at
sops
PK id (ULID)
FK company_id
title / file_path
status
sop_signoffs
FK sop_id
FK user_id
signed_at
audits
PK id (ULID)
FK company_id
FK auditor_id
status / score
feedback
submitted_at
reviewed_at
document_templates
PK id (ULID)
name / category
required (bool)
expiry_required
sample_file_path
documents
PK id (ULID)
FK company_id
FK template_id
FK uploaded_by (user)
file_path / version
status / expiry_date
audit_documents
FK audit_id
FK document_id
companies
PK id (ULID)
FK package_id
name
registration_no
status
compliance_score
journeys
PK id (ULID)
FK company_id
current_level
status
activated_at
journey_levels
PK id (ULID)
FK journey_id
level (L1-L4)
required_score
status
unlocked_at
milestones
PK id (ULID)
FK journey_level_id
name / order
status
trust_badges
PK id (ULID)
FK company_id
level
issued_at
certificate_path
packages
PK id (ULID)
name / price
duration_days
storage_limit
user_limit
features (json)
payments
PK id (ULID)
FK company_id
FK package_id
amount / method
status
invoice_no
paid_at
1:N
1:1
uploaded_by

5.1 Entity Dictionary

Entity	Purpose
companies	Tenant record for each company on the platform; holds status, current compliance score, and linked subscription package.
users	All platform users (admin, staff, finance, auditor accounts) scoped to a company (nullable for internal admin/auditor accounts) with a role.
roles / permissions / role_permission	RBAC configuration — flexible role definitions without hard-coding access rules in application code.
packages	Subscription plans: pricing, duration, storage & user limits, and enabled feature flags (JSON).
payments	Payment/subscription transactions per company; drives package activation on approval.
journeys / journey_levels / milestones	The guided compliance journey: one journey per company, containing ordered levels (L1–L4) unlocked by score, each with milestones.
document_templates	Admin-configured document requirements (required/optional, expiry rules) that drive what a company must upload.
documents	Company-uploaded files against a template, with version and verification status.
auditors / audits / audit_documents	Third-party auditor profiles and the audits they perform against a company's submitted documents.
trust_badges	Issued badges/certificates once a journey level is approved.
notifications	Per-user in-app/email notification records.
support_tickets	Company support requests routed to the support team.
sops / sop_signoffs	Company SOP documents and the employee acknowledgment/sign-off trail.

5.2 Key Design Notes

- ULID over auto-increment: sortable by creation time (good for activity/audit feeds) while remaining non-sequential and safe to expose in API responses and URLs.
- Every foreign key is a company_id-scoped relationship where relevant, enforced at the query layer via a global Eloquent scope — this is the multi-tenancy boundary that keeps one company's data from ever leaking into another's queries.
- Status fields (company, payment, document, audit, journey-level) are backed by PHP enums mirrored as PostgreSQL CHECK constraints, so invalid states are rejected at both the application and database layer.
- Money fields stored as integer cents (not float) to avoid rounding errors in payment/invoice calculations.
- Audit-sensitive tables (payments, audits, documents) are append-friendly and soft-deleted, never hard-deleted, to preserve a full compliance trail.

6. Landing Page — Stack, Structure & Performance

The marketing landing page is not a separate project or a separate stack — it ships from the same Next.js application, the same design system, and the same deploy pipeline as the product. This keeps the brand consistent between 'the site that sells it' and 'the product that delivers it,' and avoids maintaining a second codebase.

6.1 Stack

Layer	Technology	Purpose
Framework	Next.js (App Router) — Static Generation / ISR	Same repo as the product app, but the marketing route group is pre-rendered at build time for instant load and SEO
Styling	Tailwind CSS + shadcn/ui (shared design tokens)	Visual consistency between marketing site and product; no second design system to maintain
Animation	Framer Motion (motion)	Scroll-triggered fade/slide-ins, hover states, smooth page/section transitions
Smooth scroll	Lenis	Adds the inertia / 'premium SaaS' scroll feel modern landing pages use
Carousels / logo strips	Embla Carousel	Lightweight, accessible testimonial and partner-logo carousels
Icons	lucide-react	Same icon set already used across the product — zero extra bundle for a second icon library
Images	next/image + CDN	Automatic resizing, lazy-loading, modern formats (WebP/AVIF) for a fast-feeling hero
Forms (demo request / waitlist)	React Hook Form + Zod, posts to a Laravel API endpoint	Same validation pattern as the product app; leads land directly in the back office, not a third-party form tool
Analytics	Vercel Analytics or PostHog	Conversion tracking: hero CTA clicks, scroll depth, sign-up funnel drop-off
Optional polish	Pre-built Tailwind/shadcn-compatible sections (Aceternity UI, Magic UI, shadcn Blocks) — copied in as owned source, restyled to brand	Fast path to a 'designed by a studio' look without vendor lock-in or extra runtime dependencies

6.2 Page Structure

A single long-scroll landing page, structured around the platform flow already defined in the roadmap, so the marketing narrative and the product experience tell the same story.

Section	Purpose / Content
Hero	Core value proposition ('Comply. Scale. Lead.'), primary CTA (Get Started / Book a Demo), a lightweight animated visual of the trust-score or journey concept — not a heavy video, to protect load time.

Section	Purpose / Content
Social proof strip	Logos of pilot companies / partners / auditors, or credibility markers (frameworks supported, number of compliance checks) if logos aren't available yet.
How it works	The 6-step platform flow (Register → Profile → Package → Payment → Activation → Access) as a horizontal animated stepper — reuses the same flow already designed for the product.
Compliance journey	Visual of the Comply → Scale → Lead levels (L1–L4), tying the marketing pitch directly to the product's core mechanic.
Feature grid	The 8 core modules (User Management, Dashboard, Compliance Journey, Secure Data Center, Audit Management, Third-Party Auditor, Smart Data Room, SOP Management) as a responsive card grid.
Packages / pricing	Package tiers pulled from the same packages table the product uses — one source of truth, no hardcoded pricing drifting out of sync with the back office.
Testimonials / case study	Carousel (Embla) — swap in as pilot customers come on board; ships with placeholder-safe design so it's not empty pre-launch.
FAQ	Accordion (shadcn Accordion) — addresses compliance/security/pricing objections before they reach sales.
Final CTA + footer	Repeat primary CTA, secondary links, trust badges (security/compliance certifications once obtained), contact/footer nav.

6.3 Folder Structure (inside the existing Next.js app)

```
src/app/
├─ (marketing)/            ← public, statically generated, no auth
│   ├─ layout.tsx          (marketing-only header/footer, no dashboard chrome)
│   ├─ page.tsx            (landing page)
│   ├─ pricing/page.tsx
│   └─ opengraph-image.tsx  (auto-generated social share image)
├─ (auth)/...
├─ (backoffice)/...        (existing, auth-guarded route group)
features/
└─ marketing/
   ├─ components/          (Hero, HowItWorks, FeatureGrid, PricingTable, Faq, ...)
   ├─ animations/          (shared Framer Motion variants)
   └─ content.ts           (copy/content, kept out of components for easy editing)
```

- The (marketing) route group is fully decoupled from the (backoffice) group — different layout, no auth check overhead, no dashboard JS shipped to a visitor who hasn't signed up yet.
- Marketing components are built from the same components/ui/ shadcn primitives as the product, just composed differently — one Button, one design token set, everywhere.

6.4 Performance & SEO Checklist

- Statically generate the page at build time (or ISR with a long revalidate window) — no client-side data fetching blocking first paint.
- Only hydrate what's interactive (animations, forms, FAQ accordion); everything else ships as static HTML.
- Proper metadata export: title, description, canonical URL, and an OpenGraph/Twitter card image for clean social sharing.
- Target Lighthouse 90+ on mobile before launch — the hero's image/video is almost always the biggest offender, so it is size-capped and served via next/image.

- Semantic HTML and alt text throughout (accessibility and SEO overlap here) — headings in a real h1→h2→h3 hierarchy, not styled divs.
- Structured data (JSON-LD, Organization/Product schema) added for richer search-engine results.